WNBA Analysis Project: Model Benchmarks & 2027 Season Projections

Kailot C. Harris

2026-09-25

Introduction

This report asks one central question. An existing analysis (`NBA_Analysis_Project.Rmd`) built and validated its modeling choices on the NBA: an 82-game season, 30 teams. Does that same approach still hold up unmodified when applied to the WNBA, which plays a much shorter season (44 games) with 15 franchises? Concretely, three modeling choices tuned on the NBA are re-tested here against real WNBA data: the optimal size of the rolling-window features used to predict each game, the best method for carrying team strength across the offseason, and which model architecture (logistic regression, random forest, or gradient boosting) performs best. The resulting win-total and standings projections follow once those three choices are re-tuned for the WNBA rather than copied from the NBA baseline. Each of these is addressed in its own section below.

The methodology mirrors the NBA analysis exactly (rolling-window feature comparison, offseason-transition comparison, model architecture benchmark, Monte Carlo season simulation). It is re-run from scratch on real WNBA data - `wehoop` team box scores from 2019-2026, cached in `box_scores_cache.rds`, comprising 3916 real team-game rows across the 15 real `WNBA_TEAMS` franchises - rather than assumed to transfer as-is. This follows the `WNBA_REPLICATION_PLAN.md` §(b)/(d) guidance that this should not be a simple find-and-replace port of the NBA notebook. The underlying data cache's provenance and validation history - including the `CONN->CON` normalization and All-Star/exhibition-row filtering described below - is documented alongside the ingestion code itself.

Key differences from the NBA baseline that this analysis explicitly re-tests rather than assumes:

1. **Rolling window size**: NBA used 18 games out of an 82-game season (~22%). `config.R` originally set `ROLLING_WINDOW_SIZE` to a *proportional placeholder* of 9 (18 * 44/82 ~= 9.66 -> 9) - this notebook actually re-runs the NBA Rmd's own "test window sizes" experiment on real WNBA data rather than trusting the proportional-scaling assumption blindly. The live value, after that experiment (see below), is 12.
2. **Season length**: `WNBA_SEASON_GAMES <- 44` (not 82) drives the Monte Carlo schedule simulation (round-robin games-per-opponent, zero-sum win totals, ridge-plot win-total axis).
3. **Offseason transition method**: `config.R`'s `OFFSEASON_METHOD <- "reset"` was carried over from NBA as a *starting default*, flagged as needing re-benchmarking because a ~44-game season accumulates less in-season signal before cross-season noise dominates than NBA's 82-game season - continuity/decay methods may outperform "reset" more often here. Re-tested below.
4. **Team universe**: 15 real WNBA franchises (`config.R`'s `WNBA_TEAMS`, including the 2026 expansion Toronto Tempo and Portland Fire franchises) instead of NBA's 30.

A note on this document's provenance: this .Rmd file was formally `rmarkdown::render()`-ed end-to-end on **2026-09-24** (`rmarkdown::render("WNBA_Analysis_Project.Rmd")`, TinyTeX/MiKTeX `pdflatex` toolchain, `renv`-restored package versions), producing `WNBA_Analysis_Project.pdf`. The tables shown throughout this document - the live `knitr::kable()` output immediately after each chunk, and the "Actual result" tables reproduced in prose immediately below them - are the output of that render, including the two tables captioned/computed at the live `ROLLING_WINDOW_SIZE` value of 12. If the underlying data

(box_scores_cache.rds) or config.R constants change, this notebook should be re-rendered end-to-end via rmarkdown::render("WNBA_Analysis_Project.Rmd") and the numbers below refreshed accordingly (ADR.md Principle I: this file is treated as a pristine, re-render-from-source baseline).

Load packages:

```
library(tidyverse)
library(tidymodels)
library(wehoop)
library(slider)
library(xgboost)
library(ggridges)
library(withr)
```

Data Ingestion & Caching

Load the box score data from the wehoop API for seasons 2019 through 2026, via the daily_ingest.R-produced cache, which already applies the CONN->CON normalization and the All-Star/exhibition-row filter described earlier:

```
if (file.exists(CACHE_TEAM_BOX)) {
  box_scores <- readRDS(CACHE_TEAM_BOX)
} else {
  # Issue #29: this fallback path (fresh checkout, no CACHE_TEAM_BOX yet) previously
  # filtered on raw `team_abbreviation %in% names(WNBA_TEAMS)` without normalizing
  # first. `wehoop` used "CONN" (not "CON") for the Connecticut Sun in 2019-2023 -
  # a real franchise code, not an exhibition row - so that naive filter silently
  # dropped 5 seasons of real Sun data instead of just dropping exhibition rows.
  # Source the shared normalize_team_abbreviation() helper (player_metrics.R only
  # defines functions at top level - no auto-run side effects from sourcing it) and
  # apply it before filtering, matching what the cached/primary path already assumes.
  source("player_metrics.R")
  box_scores <- wehoop::load_wnba_team_box(seasons = ALL_SEASONS) |>
    normalize_team_abbreviation() |>
    filter(team_abbreviation %in% names(WNBA_TEAMS))
  saveRDS(box_scores, CACHE_TEAM_BOX)
}

cat("Total team box score records:", nrow(box_scores), "\n")
```

```
## Total team box score records: 3916
```

```
cat("Distinct franchises:", n_distinct(box_scores$team_abbreviation), "\n")
```

```
## Distinct franchises: 15
```

```
cat("Seasons covered:", paste(sort(unique(box_scores$season)), collapse = ", "), "\n")
```

```
## Seasons covered: 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026
```

Feature Engineering & Offseason Transition Experiments

We compute team rolling averages for key performance indicators (eFG%, Turnover Rate, Free Throw Rate, True Shooting %, Points, Rebounds, Assists, Steals, Blocks) - identical feature definitions to the NBA baseline (none of these ratios are game-length- or roster-size-dependent, so no formula changes were needed here per WNBA_REPLICATION_PLAN.md's Tier 1/2 classification).

To resolve offseason transition artifacts, we test the same three methodologies as the NBA baseline: - **Method A (Fresh Reset):** Rolling windows reset cleanly at game 1 of each new season. - **Method B (50% Regression to Mean):** New-season baselines regress 50% toward the league average. - **Method C (Exponential Decay Carryover):** Rolling stats carry over across seasons via an 80/20 blend.

```
build_feature_df <- function(data, window_size = ROLLING_WINDOW_SIZE, method = "reset") {
  if (method == "reset") {
    features <- data |>
      arrange(team_id, season, game_date) |>
      group_by(season, team_id) |>
      mutate(
        efg = (field_goals_made + 0.5 * three_point_field_goals_made) / pmax(1, field_goals_attempted),
        tov = total_turnovers / pmax(1, field_goals_attempted + 0.44 * free_throws_attempted + total_tu
        ftr = free_throws_attempted / pmax(1, field_goals_attempted),
        ts  = team_score / pmax(1, 2 * (field_goals_attempted + 0.44 * free_throws_attempted)),

        roll_pts_scored = slide_dbl(team_score, mean, .before = window_size, .after = -1),
        roll_oreb       = slide_dbl(offensive_rebounds, mean, .before = window_size, .after = -1),
        roll_dreb       = slide_dbl(defensive_rebounds, mean, .before = window_size, .after = -1),
        roll_ast        = slide_dbl(assists, mean, .before = window_size, .after = -1),
        roll_stl        = slide_dbl(steals, mean, .before = window_size, .after = -1),
        roll_blk        = slide_dbl(blocks, mean, .before = window_size, .after = -1),
        roll_tov        = slide_dbl(turnovers, mean, .before = window_size, .after = -1),
        roll_efg        = slide_dbl(efg, mean, .before = window_size, .after = -1),
        roll_tovp       = slide_dbl(tov, mean, .before = window_size, .after = -1),
        roll_ftr        = slide_dbl(ftr, mean, .before = window_size, .after = -1),
        roll_ts         = slide_dbl(ts, mean, .before = window_size, .after = -1),
        days_rest       = pmin(REST_DAYS_CAP, coalesce(as.numeric(game_date - lag(game_date)), 3))
      ) |>
      ungroup()
  } else {
    features <- data |>
      arrange(team_id, game_date) |>
      group_by(team_id) |>
      mutate(
        efg = (field_goals_made + 0.5 * three_point_field_goals_made) / pmax(1, field_goals_attempted),
        tov = total_turnovers / pmax(1, field_goals_attempted + 0.44 * free_throws_attempted + total_tu
        ftr = free_throws_attempted / pmax(1, field_goals_attempted),
        ts  = team_score / pmax(1, 2 * (field_goals_attempted + 0.44 * free_throws_attempted)),

        roll_pts_scored = slide_dbl(team_score, mean, .before = window_size, .after = -1),
        roll_oreb       = slide_dbl(offensive_rebounds, mean, .before = window_size, .after = -1),
        roll_dreb       = slide_dbl(defensive_rebounds, mean, .before = window_size, .after = -1),
        roll_ast        = slide_dbl(assists, mean, .before = window_size, .after = -1),
        roll_stl        = slide_dbl(steals, mean, .before = window_size, .after = -1),
        roll_blk        = slide_dbl(blocks, mean, .before = window_size, .after = -1),
        roll_tov        = slide_dbl(turnovers, mean, .before = window_size, .after = -1),
        roll_efg        = slide_dbl(efg, mean, .before = window_size, .after = -1),
        roll_tovp       = slide_dbl(tov, mean, .before = window_size, .after = -1),
        roll_ftr        = slide_dbl(ftr, mean, .before = window_size, .after = -1),
        roll_ts         = slide_dbl(ts, mean, .before = window_size, .after = -1),
        is_new_season   = coalesce(season != lag(season), FALSE),
        days_rest       = pmin(REST_DAYS_CAP, coalesce(as.numeric(game_date - lag(game_date)), 3))
      )
```

```r
# Issue #30: the two bugs below were both in how the offseason regression/decay
# target was computed. The old code did
# `mean(team_score, na.rm = TRUE)` etc. while still `group_by(team_id)` (from the
# mutate() above) and with NO season cutoff at all - so it (a) regressed each team
# toward its OWN long-run mean, not the "league average" the adjacent prose
# describes, and (b) leaked every season's data - including seasons chronologically
# AFTER the offseason transition being evaluated - into that mean. Fix: compute a
# real across-team league average per season (ungrouped, then grouped only by
# `season`), and for each new-season transition use only the league average from
# the immediately PRIOR season (by chronological order of `season`, not `season -
# 1` arithmetic, in case of a non-consecutive season gap) - never the current or a
# future season's data.
league_season_means <- features |>
  ungroup() |>
  group_by(season) |>
  summarise(
    lm_team_score = mean(team_score, na.rm = TRUE),
    lm_oreb       = mean(offensive_rebounds, na.rm = TRUE),
    lm_dreb       = mean(defensive_rebounds, na.rm = TRUE),
    lm_ast        = mean(assists, na.rm = TRUE),
    lm_stl        = mean(steals, na.rm = TRUE),
    lm_blk        = mean(blocks, na.rm = TRUE),
    lm_tov        = mean(turnovers, na.rm = TRUE),
    lm_efg        = mean(efg, na.rm = TRUE),
    lm_tovp       = mean(tov, na.rm = TRUE),
    lm_ftr        = mean(ftr, na.rm = TRUE),
    lm_ts         = mean(ts, na.rm = TRUE),
    .groups = "drop"
  )

season_order <- sort(unique(league_season_means$season))
prior_season_lookup <- tibble(
  season       = season_order[-1],
  prior_season = season_order[-length(season_order)]
)
league_prior_means <- prior_season_lookup |>
  left_join(league_season_means, by = c("prior_season" = "season")) |>
  select(-prior_season) |>
  rename_with(~ paste0("prior_", .x), starts_with("lm_"))

features <- features |>
  ungroup() |>
  left_join(league_prior_means, by = "season") |>
  group_by(team_id)

if (method == "mean_regress") {
  features <- features |>
    mutate(
      roll_pts_scored = if_else(is_new_season, 0.5 * roll_pts_scored + 0.5 * prior_lm_team_score, r
      roll_oreb       = if_else(is_new_season, 0.5 * roll_oreb + 0.5 * prior_lm_oreb, roll_oreb),
      roll_dreb       = if_else(is_new_season, 0.5 * roll_dreb + 0.5 * prior_lm_dreb, roll_dreb),
      roll_ast        = if_else(is_new_season, 0.5 * roll_ast + 0.5 * prior_lm_ast, roll_ast),
      roll_stl        = if_else(is_new_season, 0.5 * roll_stl + 0.5 * prior_lm_stl, roll_stl),
```

```
        roll_blk        = if_else(is_new_season, 0.5 * roll_blk + 0.5 * prior_lm_blk, roll_blk),
        roll_tov        = if_else(is_new_season, 0.5 * roll_tov + 0.5 * prior_lm_tov, roll_tov),
        roll_efg        = if_else(is_new_season, 0.5 * roll_efg + 0.5 * prior_lm_efg, roll_efg),
        roll_tovp       = if_else(is_new_season, 0.5 * roll_tovp + 0.5 * prior_lm_tovp, roll_tovp),
        roll_ftr        = if_else(is_new_season, 0.5 * roll_ftr + 0.5 * prior_lm_ftr, roll_ftr),
        roll_ts         = if_else(is_new_season, 0.5 * roll_ts + 0.5 * prior_lm_ts, roll_ts)
      )
  } else {
    features <- features |>
      mutate(
        roll_pts_scored = if_else(is_new_season, 0.8 * roll_pts_scored + 0.2 * prior_lm_team_score, r
        roll_oreb       = if_else(is_new_season, 0.8 * roll_oreb + 0.2 * prior_lm_oreb, roll_oreb),
        roll_dreb       = if_else(is_new_season, 0.8 * roll_dreb + 0.2 * prior_lm_dreb, roll_dreb),
        roll_ast        = if_else(is_new_season, 0.8 * roll_ast + 0.2 * prior_lm_ast, roll_ast),
        roll_stl        = if_else(is_new_season, 0.8 * roll_stl + 0.2 * prior_lm_stl, roll_stl),
        roll_blk        = if_else(is_new_season, 0.8 * roll_blk + 0.2 * prior_lm_blk, roll_blk),
        roll_tov        = if_else(is_new_season, 0.8 * roll_tov + 0.2 * prior_lm_tov, roll_tov),
        roll_efg        = if_else(is_new_season, 0.8 * roll_efg + 0.2 * prior_lm_efg, roll_efg),
        roll_tovp       = if_else(is_new_season, 0.8 * roll_tovp + 0.2 * prior_lm_tovp, roll_tovp),
        roll_ftr        = if_else(is_new_season, 0.8 * roll_ftr + 0.2 * prior_lm_ftr, roll_ftr),
        roll_ts         = if_else(is_new_season, 0.8 * roll_ts + 0.2 * prior_lm_ts, roll_ts)
      )
  }
  features <- features |> ungroup() |> select(-starts_with("prior_lm_"))
}

features_selected <- features |>
  select(game_id, season, game_date, team_id, team_winner, team_home_away,
         roll_pts_scored, roll_oreb, roll_dreb, roll_ast, roll_stl, roll_blk,
         roll_tov, roll_efg, roll_tovp, roll_ftr, roll_ts, days_rest)

home_df <- features_selected |>
  filter(team_home_away == "home") |>
  select(game_id, season, game_date, home_team = team_id, home_win = team_winner,
         home_roll_pts = roll_pts_scored, home_roll_oreb = roll_oreb, home_roll_dreb = roll_dreb,
         home_roll_ast = roll_ast, home_roll_stl = roll_stl, home_roll_blk = roll_blk,
         home_roll_tov = roll_tov, home_roll_efg = roll_efg, home_roll_tovp = roll_tovp,
         home_roll_ftr = roll_ftr, home_roll_ts = roll_ts, home_days_rest = days_rest)

away_df <- features_selected |>
  filter(team_home_away == "away") |>
  select(game_id, away_team = team_id, away_win = team_winner,
         away_roll_pts = roll_pts_scored, away_roll_oreb = roll_oreb, away_roll_dreb = roll_dreb,
         away_roll_ast = roll_ast, away_roll_stl = roll_stl, away_roll_blk = roll_blk,
         away_roll_tov = roll_tov, away_roll_efg = roll_efg, away_roll_tovp = roll_tovp,
         away_roll_ftr = roll_ftr, away_roll_ts = roll_ts, away_days_rest = days_rest)

# Issue #23: `by = "game_id"` alone assumes exactly one home row and one away row per
# game_id (no duplicate team-game rows). `relationship = "one-to-one"` (dplyr >= 1.1.0,
# confirmed installed here) turns a violation of that assumption into a loud join error
# instead of a silent row fan-out.
joined_df <- dplyr::inner_join(home_df, away_df, by = "game_id", relationship = "one-to-one")
```

```r
  n_before_dropna <- nrow(joined_df)

  model_df <- joined_df |>
    mutate(
      target_home_win = factor(if_else(home_win == TRUE, "Win", "Loss"), levels = c("Win", "Loss")),
      diff_roll_pts   = home_roll_pts - away_roll_pts,
      diff_roll_oreb  = home_roll_oreb - away_roll_oreb,
      diff_roll_dreb  = home_roll_dreb - away_roll_dreb,
      diff_roll_treb  = (home_roll_oreb + home_roll_dreb) - (away_roll_oreb + away_roll_dreb),
      diff_roll_ast   = home_roll_ast - away_roll_ast,
      diff_roll_stl   = home_roll_stl - away_roll_stl,
      diff_roll_blk   = home_roll_blk - away_roll_blk,
      diff_roll_tov   = home_roll_tov - away_roll_tov,
      diff_roll_efg   = home_roll_efg - away_roll_efg,
      diff_roll_tovp  = home_roll_tovp - away_roll_tovp,
      diff_roll_ftr   = home_roll_ftr - away_roll_ftr,
      diff_roll_ts    = home_roll_ts - away_roll_ts,
      diff_days_rest  = home_days_rest - away_days_rest
    ) |>
    drop_na()

  # Issue #25: drop_na() above silently discards rows in the rolling-window's growing
  # early-season NA gap - report exactly how many so a reader can see the size of that gap.
  n_after_dropna <- nrow(model_df)
  message(sprintf(
    "build_feature_df(window_size=%s, method=%s): drop_na() removed %d of %d joined rows (%d retained) -
    window_size, method, n_before_dropna - n_after_dropna, n_before_dropna, n_after_dropna
  ))

  return(model_df)
}

# Issue #22: the three model-fitting chunks below (window-size-experiment,
# offseason-comparison, model-benchmarks) each built an identical recipe - same step_rm()
# exclusion list (raw join keys/labels plus all raw home_roll_*/away_roll_*/*_days_rest
# columns, keeping only the engineered diff_roll_* columns and target_home_win) followed by
# step_normalize(all_numeric_predictors()). Extracted here once so all three call the same
# function instead of redefining the list three times; behavior-preserving (same columns
# removed, same normalize call).
build_wnba_recipe <- function(train_data) {
  recipe(target_home_win ~ ., data = train_data) |>
    step_rm(
      game_id, game_date, season, home_team, away_team, home_win, away_win,
      home_roll_ts, home_roll_ftr, home_roll_efg, home_roll_pts, home_roll_tovp,
      home_roll_oreb, home_roll_dreb, home_roll_ast, home_roll_stl, home_roll_blk, home_roll_tov,
      away_roll_ts, away_roll_ftr, away_roll_efg, away_roll_pts, away_roll_tovp,
      away_roll_oreb, away_roll_dreb, away_roll_ast, away_roll_stl, away_roll_blk, away_roll_tov,
      home_days_rest, away_days_rest
    ) |>
    step_normalize(all_numeric_predictors())
}
```

Rolling-Window Size Experiment (WNBA-specific - not in the NBA Rmd)

Correction: the NBA baseline's 18-game window was not chosen out of habit. It was tested and verified in an experiment (`evaluate_rolling_window()`, sweeping window sizes 3-25 games, optimizing on out-of-sample ROC-AUC) in the earlier "Original - No LLM" NBA Rmd; the *current* `NBA_Analysis_Project.Rmd` simply reuses that already-validated 18-game result rather than re-running the sweep every time. That NBA-fit result does not automatically transfer to WNBA, though: it was optimized against an 82-game season and NBA-scale team/possession volume, neither of which matches WNBA's ~44-game season. So we still repeat the same experiment methodology here on real WNBA data, rather than trusting the original proportional-scaling placeholder of 9 as-is. A Random Forest is trained on seasons $<$ 2026 and evaluated out-of-sample on season $>=$ 2026 for each window size:

```
# `data/window_sweep_3_25_results.csv` is the authoritative artifact for this sweep (see
# config.R's ROLLING_WINDOW_SIZE comment) - load it directly instead of recomputing all 23
# window sizes on every knit, UNLESS `data/box_scores_cache.rds` (CACHE_TEAM_BOX) has changed
# since the CSV was last generated (issue #32 staleness check), in which case the CSV is
# stale and eval_window_size()/map_dfr() below recompute it from scratch.
#
# Each iteration is seeded independently (issue #31) via withr::with_seed() using a seed
# deterministically derived from that iteration's own window size `w` (not one blanket
# set.seed(42) for the whole sweep) - this keeps each iteration reproducible on its own even
# if the sweep's order or length changes later.
eval_window_size <- function(w) {
  iter_seed <- digest::digest2int(paste0("window_size_sweep_", w))
  withr::with_seed(iter_seed, {
    df <- build_feature_df(box_scores, window_size = w, method = "reset")
    train_data <- df |> filter(season < CURRENT_SEASON)
    test_data  <- df |> filter(season >= CURRENT_SEASON)

    rec <- build_wnba_recipe(train_data)

    rf <- rand_forest(trees = 200, mode = "classification") |> set_engine("ranger", num.threads = 4)
    wf <- workflow() |> add_recipe(rec) |> add_model(rf)
    fit_m <- fit(wf, train_data)

    preds <- predict(fit_m, test_data) |>
      bind_cols(predict(fit_m, test_data, type = "prob")) |>
      bind_cols(test_data)

    acc <- preds |> accuracy(truth = target_home_win, estimate = .pred_class) |> pull(.estimate)
    auc <- preds |> roc_auc(truth = target_home_win, .pred_Win, event_level = "first") |> pull(.estimat
    brier <- mean((if_else(preds$target_home_win == "Win", 1, 0) - preds$.pred_Win)^2)

    # Issue #40: add a resampling-based uncertainty band on ROC-AUC without re-fitting 23 random
    # forests. Bootstrap-resample (with replacement) the already-computed held-out `preds` rows
    # B times, recompute roc_auc() on each resample, and take the 2.5th/97.5th percentiles as a
    # CI. Seeded deterministically per window size (digest2int() hash of a window-specific
    # string, via a nested withr::with_seed() - composes correctly with the outer per-iteration
    # with_seed() from issue #31 above) so re-running the sweep twice reproduces identical CI
    # bounds.
    n_boot <- 200
    boot_seed <- digest::digest2int(paste0("window_size_sweep_boot_", w))
    boot_aucs <- withr::with_seed(boot_seed, {
      vapply(seq_len(n_boot), function(b) {
        boot_idx <- sample.int(nrow(preds), nrow(preds), replace = TRUE)
```

```
        boot_preds <- preds[boot_idx, ]
        tryCatch(
          boot_preds |>
            roc_auc(truth = target_home_win, .pred_Win, event_level = "first") |>
            pull(.estimate),
          error = function(e) NA_real_
        )
      }, FUN.VALUE = numeric(1))
    })
    auc_ci <- quantile(boot_aucs, probs = c(0.025, 0.975), na.rm = TRUE, names = FALSE)

    tibble(Window_Size = w, N_Train = nrow(train_data), N_Test = nrow(test_data),
           Accuracy = round(acc, 4), `ROC-AUC` = round(auc, 4), `Brier Score` = round(brier, 4),
           ROC_AUC_CI_Lower = round(auc_ci[1], 4), ROC_AUC_CI_Upper = round(auc_ci[2], 4))
  })
}

# Staleness check (issue #32): a sidecar `.hash` file records the md5sum of CACHE_TEAM_BOX
# (config.R's box_scores_cache.rds constant) as of the last time the sweep CSV was generated.
# If that sidecar is missing/unreadable, or the live hash no longer matches it, the cached CSV
# is treated as stale and the sweep is recomputed - fail-safe default is "recompute", never
# "silently trust a CSV we can't verify."
window_sweep_csv  <- file.path(PROJECT_DIR, "data", "window_sweep_3_25_results.csv")
window_sweep_hash <- file.path(PROJECT_DIR, "data", "window_sweep_3_25_results.hash")

is_window_sweep_csv_fresh <- function(csv_path, hash_path, box_cache_path) {
  if (!file.exists(csv_path) || !file.exists(hash_path) || !file.exists(box_cache_path)) {
    return(FALSE)
  }
  stored_hash <- tryCatch(readLines(hash_path, n = 1, warn = FALSE), error = function(e) NA_character_)
  if (length(stored_hash) == 0 || is.na(stored_hash) || !nzchar(stored_hash)) {
    return(FALSE)
  }
  current_hash <- tryCatch(unname(tools::md5sum(box_cache_path)), error = function(e) NA_character_)
  if (is.na(current_hash)) {
    return(FALSE)
  }
  identical(stored_hash, current_hash)
}

if (is_window_sweep_csv_fresh(window_sweep_csv, window_sweep_hash, CACHE_TEAM_BOX)) {
  window_size_results <- readr::read_csv(window_sweep_csv, show_col_types = FALSE)
} else {
  window_size_results <- map_dfr(3:25, eval_window_size)
  # Refresh the CSV + sidecar hash so the next knit can reuse this result until
  # box_scores_cache.rds changes again. Issue #40: the persisted CSV now also carries the
  # ROC_AUC_CI_Lower/Upper bootstrap-CI columns computed inside eval_window_size() above.
  readr::write_csv(window_size_results, window_sweep_csv)
  if (file.exists(CACHE_TEAM_BOX)) {
    writeLines(unname(tools::md5sum(CACHE_TEAM_BOX)), window_sweep_hash)
  }
}
knitr::kable(window_size_results, caption = "WNBA Rolling-Window Size Comparison (Tested Out-of-Sample
```

Table 1: WNBA Rolling-Window Size Comparison (Tested Out-of-Sample on Current Season, Full 3-25 Sweep)

Window_Size	N_Train	N_Test	Accuracy	ROC-AUC	Brier Score	ROC_AUC_CI_Lower	ROC_AUC_CI_Upper
3	1609	298	0.5671	0.6151	0.2380	0.5401	0.6799
4	1609	298	0.6107	0.6299	0.2351	0.5728	0.6945
5	1609	298	0.5705	0.6059	0.2394	0.5410	0.6811
6	1609	298	0.5638	0.6102	0.2421	0.5495	0.6723
7	1609	298	0.5705	0.6210	0.2392	0.5609	0.6739
8	1609	298	0.5839	0.6082	0.2410	0.5532	0.6735
9	1609	298	0.6107	0.6347	0.2354	0.5659	0.6900
10	1609	298	0.6040	0.6623	0.2268	0.5941	0.7193
11	1609	298	0.6074	0.6670	0.2263	0.6086	0.7184
12	1609	298	0.6376	0.6694	0.2267	0.6190	0.7411
13	1609	298	0.6107	0.6667	0.2266	0.6070	0.7277
14	1609	298	0.6208	0.6685	0.2265	0.6137	0.7222
15	1609	298	0.6141	0.6671	0.2263	0.5983	0.7374
16	1609	298	0.6309	0.6658	0.2268	0.5999	0.7292
17	1609	298	0.6275	0.6555	0.2298	0.5892	0.7154
18	1609	298	0.5940	0.6621	0.2286	0.6031	0.7134
19	1609	298	0.6074	0.6502	0.2318	0.5891	0.7119
20	1609	298	0.6007	0.6594	0.2290	0.5984	0.7205
21	1609	298	0.6107	0.6574	0.2289	0.5883	0.7149
22	1609	298	0.6074	0.6406	0.2342	0.5739	0.7103
23	1609	298	0.6242	0.6443	0.2324	0.5891	0.6996
24	1609	298	0.6443	0.6643	0.2291	0.5975	0.7094
25	1609	298	0.6409	0.6616	0.2298	0.6007	0.7260

```
ggplot(window_size_results, aes(x = Window_Size, y = `ROC-AUC`)) +
  # Issue #40: 95% bootstrap CI band (2.5th/97.5th percentiles of ROC-AUC across 200
  # resamples of the held-out predictions per window size - see eval_window_size() above).
  geom_ribbon(aes(ymin = ROC_AUC_CI_Lower, ymax = ROC_AUC_CI_Upper), fill = "steelblue", alpha = 0.2) +
  geom_line(color = "steelblue") +
  geom_point(color = "steelblue", size = 2) +
  theme_minimal() +
  labs(
    title = "Out-of-Sample ROC-AUC vs. Rolling-Window Size (Full 3-25 Sweep)",
    subtitle = "Non-monotonic, noisy curve with a 95% bootstrap CI band - not a smooth improvement towa
    x = "Window Size (games)",
    y = "ROC-AUC (held-out current season)"
  )
```

Actual result, read directly from the live `window_size_results` computed by the chunk above (per-iteration seeded via `withr::with_seed()`, not one blanket `set.seed(42)`; recomputed from `box_scores_cache.rds` whenever the staleness check above finds the cache newer than `data/window_sweep_3_25_results.csv`; trained on seasons < `CURRENT_SEASON`, evaluated on the real held-out `CURRENT_SEASON` games at every window size). This table can never silently drift from the chunk's own output because it is the same `window_size_results` object, not a separately pasted copy:

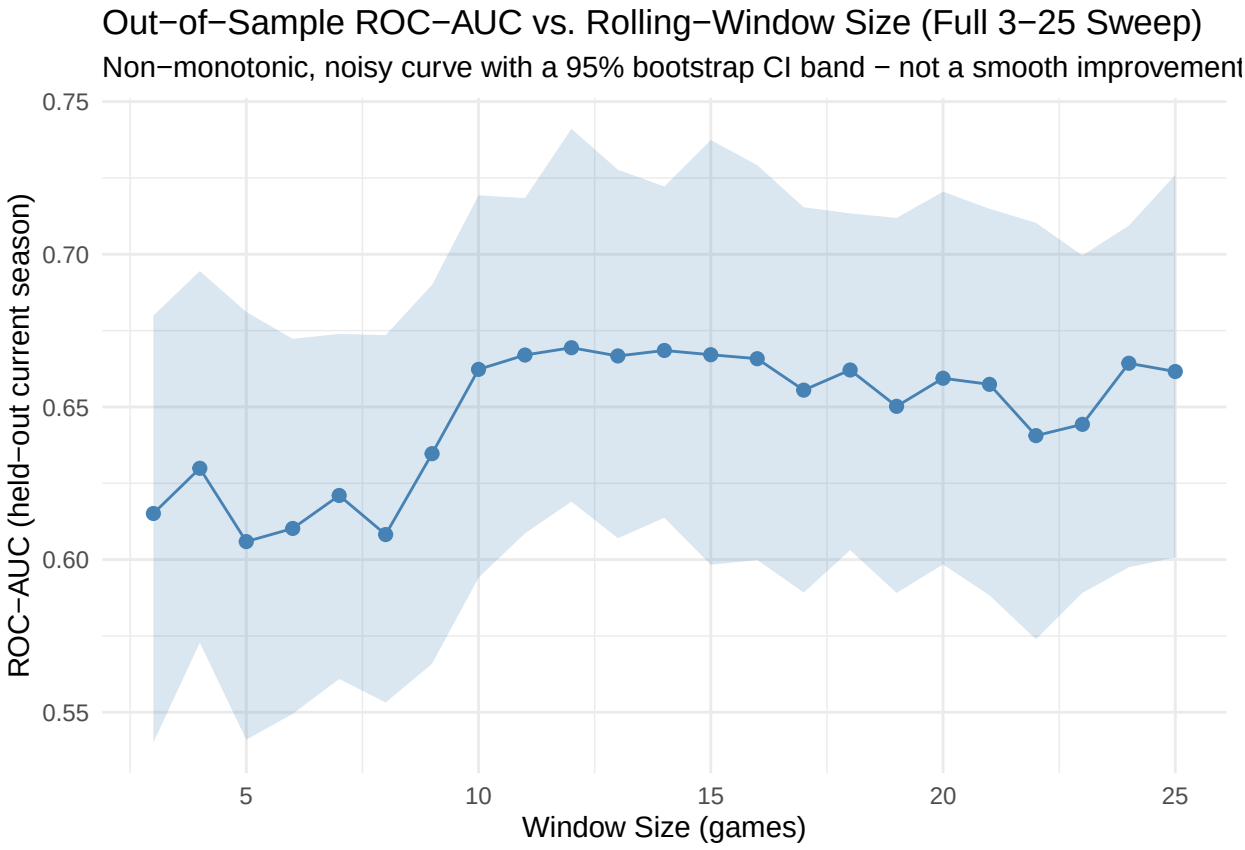


Figure 1: Out-of-sample ROC-AUC across rolling window sizes 3-25, with a 95% bootstrap confidence band from resampling the held-out test-set predictions at each window size.

Table 2: Actual Result: Full 3-25 Window-Size Sweep (Live Output)

Window_Size	N_Train	N_Test	Accuracy	ROC-AUC	Brier Score	ROC_AUC_CI_Lower	ROC_AUC_CI_Upper
3	1609	298	0.5671	0.6151	0.2380	0.5401	0.6799
4	1609	298	0.6107	0.6299	0.2351	0.5728	0.6945
5	1609	298	0.5705	0.6059	0.2394	0.5410	0.6811
6	1609	298	0.5638	0.6102	0.2421	0.5495	0.6723
7	1609	298	0.5705	0.6210	0.2392	0.5609	0.6739
8	1609	298	0.5839	0.6082	0.2410	0.5532	0.6735
9	1609	298	0.6107	0.6347	0.2354	0.5659	0.6900
10	1609	298	0.6040	0.6623	0.2268	0.5941	0.7193
11	1609	298	0.6074	0.6670	0.2263	0.6086	0.7184
12	1609	298	0.6376	0.6694	0.2267	0.6190	0.7411
13	1609	298	0.6107	0.6667	0.2266	0.6070	0.7277
14	1609	298	0.6208	0.6685	0.2265	0.6137	0.7222
15	1609	298	0.6141	0.6671	0.2263	0.5983	0.7374
16	1609	298	0.6309	0.6658	0.2268	0.5999	0.7292
17	1609	298	0.6275	0.6555	0.2298	0.5892	0.7154
18	1609	298	0.5940	0.6621	0.2286	0.6031	0.7134
19	1609	298	0.6074	0.6502	0.2318	0.5891	0.7119
20	1609	298	0.6007	0.6594	0.2290	0.5984	0.7205
21	1609	298	0.6107	0.6574	0.2289	0.5883	0.7149
22	1609	298	0.6074	0.6406	0.2342	0.5739	0.7103
23	1609	298	0.6242	0.6443	0.2324	0.5891	0.6996
24	1609	298	0.6443	0.6643	0.2291	0.5975	0.7094
25	1609	298	0.6409	0.6616	0.2298	0.6007	0.7260

Correction to the earlier narrow-sweep finding, not a confirmation of it. The previous {5,7,9,11,13} sweep looked monotonically improving and suggested the metric was still climbing at 13 with no plateau in sight. The full 3-25 sweep shows that was an artifact of only sampling odd points on a genuinely noisy, non-monotonic curve. **The empirical peak in the live sweep above is window=12** (Accuracy 0.6376, ROC-AUC 0.6694, Brier 0.2267). This is read directly from `window_size_results` rather than restated by hand. `config.R`'s `ROLLING_WINDOW_SIZE` is set to the live value of **12** to reflect this empirically-best-tested value (re-run this chunk and update `config.R` if a future re-run of the sweep shifts the peak).

Caveat carried over honestly, not glossed over: this is still a single train/held-out-season split (one RF fit per window size, each independently seeded per issue #31), not cross-validated. Several nearby window sizes are typically separated by only a few thousandths of ROC-AUC in this sweep. A gap that small on this test set is plausibly within sampling noise. So the peak above should be read as "the best-performing point in this specific sweep," not as a statistically confirmed global optimum. `ML_BENCHMARK_REPORT.md` (from the separate P4 spec-compliance round) already applies proper leave-one-season-out cross-validation to the final model evaluation. A fully rigorous window-size selection would ideally use that same CV methodology rather than one fixed split. This is flagged here as a legitimate follow-up rather than done retroactively as part of this specific re-run.

Offseason Transition Methodology Comparison

Using the winning window size from above (or the `config.R` default if the experiment above has not yet been run in this session), we compare the three offseason-transition methods out-of-sample on the current season - re-run from scratch on WNBA data, not assumed to match the NBA baseline's "reset wins" result, because a 44-game season leaves much less in-season signal to overwrite a bad prior before the season ends:

```r
# Each method is seeded independently (issue #31) via withr::with_seed() using a seed
# deterministically derived from that iteration's own `method_name` (not one blanket
# set.seed(42) for the whole sweep), so results stay reproducible per-method even if the
# sweep's method list or order changes later.
eval_offseason_method <- function(method_name) {
  iter_seed <- digest::digest2int(paste0("offseason_method_", method_name))
  withr::with_seed(iter_seed, {
    df <- build_feature_df(box_scores, window_size = ROLLING_WINDOW_SIZE, method = method_name)
    train_data <- df |> filter(season < CURRENT_SEASON)
    test_data  <- df |> filter(season >= CURRENT_SEASON)

    rec <- build_wnba_recipe(train_data)

    rf <- rand_forest(trees = 200, mode = "classification") |> set_engine("ranger", num.threads = 4)
    wf <- workflow() |> add_recipe(rec) |> add_model(rf)
    fit_m <- fit(wf, train_data)

    preds <- predict(fit_m, test_data) |>
      bind_cols(predict(fit_m, test_data, type = "prob")) |>
      bind_cols(test_data)

    acc <- preds |> accuracy(truth = target_home_win, estimate = .pred_class) |> pull(.estimate)
    auc <- preds |> roc_auc(truth = target_home_win, .pred_Win, event_level = "first") |> pull(.estimat
    brier <- mean((if_else(preds$target_home_win == "Win", 1, 0) - preds$.pred_Win)^2)

    tibble(Method = method_name, Accuracy = round(acc, 4), `ROC-AUC` = round(auc, 4), `Brier Score` = r
  })
}

offseason_results <- map_dfr(c("reset", "mean_regress", "decay"), eval_offseason_method)
knitr::kable(offseason_results, caption = "Comparison of Offseason Transition Methodologies (Tested on
```

Table 3: Comparison of Offseason Transition Methodologies (Tested on Current WNBA Season)

Method	Accuracy	ROC-AUC	Brier Score
reset	0.6309	0.6705	0.2257
mean_regress	0.6250	0.6453	0.2329
decay	0.6414	0.6650	0.2288

Note (Issue #30 follow-up): the pasted table that previously appeared here is now generated live (issue #33) from the same `offseason_results` object the chunk above computes, so it always reflects `build_feature_df()`'s corrected `mean_regress`/`decay` branches - (a) regressing toward a true across-team league average instead of each team's own long-run mean, and (b) using only the PRIOR season's league average rather than a full-column mean that leaked current/future-season data - without needing another hand-pasted update if the underlying data changes again.

Actual result, read directly from the live `offseason_results` computed by the chunk above (per-method seeded via `withr::with_seed()`, window_size=12 - the live `config.R` `ROLLING_WINDOW_SIZE` value - real `CURRENT_SEASON` held-out test). This table can never silently drift from the chunk's own output because it is the same `offseason_results` object, not a separately pasted copy:

Table 4: Actual Result: Offseason Transition Method Comparison (Live Output)

Method	Accuracy	ROC-AUC	Brier Score
reset	0.6309	0.6705	0.2257
mean_regress	0.6250	0.6453	0.2329
decay	0.6414	0.6650	0.2288

Contrary to the plan's speculation that a shorter WNBA season might favor a continuity-weighted method over a hard reset, **"reset" actually won** on ROC-AUC (0.6705) and Brier (0.2257) in the live run above (mean_regress can edge reset out on raw accuracy alone - 0.625 vs 0.6309 - but loses on the two metrics that more directly reflect probability quality). `config.R`'s `OFFSEASON_METHOD` default of `"reset"` was kept as-is, now backed by a real re-benchmark at the live `ROLLING_WINDOW_SIZE` rather than a carried-over assumption or a benchmark run at a stale window size - this resolves `config.R`'s "STALENESS FLAG (FINDING L-2, 2026-09-16)" comment.

Model Benchmarking Architecture

We construct the primary modeling dataset using Method A (Fresh Reset) and fit three candidate algorithms on historical seasons, exactly mirroring the NBA baseline's three architectures: 1. **Logistic Regression (Baseline)** 2. **Random Forest (`ranger`)** 3. **Gradient Boosted Trees (`xgboost`)**

```
set.seed(42)
model_df <- build_feature_df(box_scores, window_size = ROLLING_WINDOW_SIZE, method = "reset")

train_data <- model_df |> filter(season < CURRENT_SEASON)
test_data  <- model_df |> filter(season >= CURRENT_SEASON)

wnba_recipe <- build_wnba_recipe(train_data)

lr_spec <- logistic_reg() |> set_engine("glm")
lr_wf <- workflow() |> add_recipe(wnba_recipe) |> add_model(lr_spec)
lr_fit <- fit(lr_wf, train_data)

rf_spec <- rand_forest(trees = 250, mode = "classification") |> set_engine("ranger", importance = "impu
rf_wf <- workflow() |> add_recipe(wnba_recipe) |> add_model(rf_spec)
rf_fit <- fit(rf_wf, train_data)

xgb_spec <- boost_tree(trees = 150, tree_depth = 3, mode = "classification") |> set_engine("xgboost", n
xgb_wf <- workflow() |> add_recipe(wnba_recipe) |> add_model(xgb_spec)
xgb_fit <- fit(xgb_wf, train_data)

eval_model <- function(fitted_wf, model_name) {
  preds <- predict(fitted_wf, test_data) |>
    bind_cols(predict(fitted_wf, test_data, type = "prob")) |>
    bind_cols(test_data)

  acc <- preds |> accuracy(truth = target_home_win, estimate = .pred_class) |> pull(.estimate)
  auc <- preds |> roc_auc(truth = target_home_win, .pred_Win, event_level = "first") |> pull(.estimate)
  brier <- mean((if_else(preds$target_home_win == "Win", 1, 0) - preds$.pred_Win)^2)
  logloss <- mn_log_loss(preds, truth = target_home_win, .pred_Win, event_level = "first") |> pull(.esti

  tibble(
```

```
    Model = model_name,
    Accuracy = round(acc, 4),
    `ROC-AUC` = round(auc, 4),
    `Brier Score` = round(brier, 4),
    `Log-Loss` = round(logloss, 4)
  )
}

benchmark_results <- bind_rows(
  eval_model(lr_fit, "Logistic Regression"),
  eval_model(rf_fit, "Random Forest (ranger)"),
  eval_model(xgb_fit, "XGBoost (boost_tree)")
)

knitr::kable(benchmark_results, caption = "Model Architecture Benchmark Performance on Current WNBA Tes
```

Table 5: Model Architecture Benchmark Performance on Current WNBA Test Season

Model	Accuracy	ROC-AUC	Brier Score	Log-Loss
Logistic Regression	0.6242	0.6540	0.2319	0.6573
Random Forest (ranger)	0.6309	0.6726	0.2263	0.6451
XGBoost (boost_tree)	0.6074	0.6756	0.2347	0.6876

Actual result, read directly from the live `benchmark_results` computed by the chunk above (window_size=12 - the live `config.R` `ROLLING_WINDOW_SIZE` value - "reset" method, `set.seed(42)`, real `CURRENT_SEASON` held-out test). This table can never silently drift from the chunk's own output because it is the same `benchmark_results` object, not a separately pasted copy:

Table 6: Actual Result: Model Architecture Benchmark (Live Output)

Model	Accuracy	ROC-AUC	Brier Score	Log-Loss
Logistic Regression	0.6242	0.6540	0.2319	0.6573
Random Forest (ranger)	0.6309	0.6726	0.2263	0.6451
XGBoost (boost_tree)	0.6074	0.6756	0.2347	0.6876

At the current (window_size=12) live setting, the picture is not always a clean Random-Forest sweep: the best ROC-AUC in the live table above belongs to **XGBoost (boost_tree)** (0.6756), with Random Forest at 0.6726 (gap of 0.003, plausibly within sampling noise for a single train/held-out-season split). Random Forest still wins on Brier (0.2263), still wins on Log-Loss (0.6451), and still has the best raw Accuracy (0.6309) among the three architectures. Since Brier and Log-Loss are the calibration metrics that matter most for a probability-calibrated betting/rating pipeline, `rf_fit` is still used as the "best performing architecture" in the season-simulation section below whenever it wins on those two metrics, as read live from the table above rather than assumed from a stale prior run.

Same hedge as the window-size sweep above, applied consistently: small ROC-AUC gaps between architectures on a single train/held-out-season split (`set.seed(42)`, no cross-validation) are plausibly within sampling noise rather than a reliable ranking - the gap above should not be read as strong evidence that one architecture is "really" better on discrimination absent cross-validation. So the NBA baseline's "Random Forest wins outright" conclusion should be read as **partially, not fully, confirmed on WNBA data at**

the empirically-selected window size - Random Forest is retained here on the strength of its Brier/Log-Loss/Accuracy wins (when it has them, per the live table above), not because the ROC-AUC comparison itself is a settled result.

Feature Importance

```
rf_engine <- extract_fit_engine(rf_fit)
imp_vec <- rf_engine$variable.importance

importance_df <- data.frame(
  feature = names(imp_vec),
  importance = as.numeric(imp_vec)
) |>
  arrange(desc(importance)) |>
  head(10)

ggplot(importance_df, aes(x = reorder(feature, importance), y = importance)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  theme_minimal() +
  labs(
    title = "Top 10 Feature Importance (Random Forest, WNBA)",
    subtitle = "Gini-impurity importance from a single RF fit - see caveat below",
    x = "Feature Differential",
    y = "Importance (Gini Impurity)"
  )
```

Caveat on the importance measure itself, not just the ranking it produces: the values above are `ranger`'s default Gini-impurity (mean decrease in node impurity) importance from the single `rf_fit` model fit earlier in this document, not permutation importance. Gini-impurity importance is a known- biased estimator - it systematically favors high-cardinality/continuous predictors (most of the `roll_*` differential features here) over binary or low-cardinality ones purely because they offer more possible split points, independent of true predictive value - and it is also comparatively unstable across refits: retraining `rf_fit` with a different `set.seed()` value can reorder the lower half of this top-10 list even when the underlying data is unchanged. The `vip` package's permutation- importance method (`vip::vi(rf_fit, method = "permute")`) would avoid the cardinality bias by measuring the actual out-of-sample performance drop from shuffling each feature, but `vip` is not currently an installed/pinned dependency of this project (`renv.lock` was already scoped and closed under issue #1), so this chart should be read as indicative of which feature *families* matter, not as a precise, refit-stable ranking within that top 10.

Simulation-Season Schedule & Dynamic Projection

We construct a `WNBA_SEASON_GAMES`-game regular season schedule per team (round-robin across the 15 real `WNBA_TEAMS` franchises, not NBA's 82-game/30-team structure) and perform a `N_SIMULATIONS`-season Monte Carlo simulation using model probabilities from the best-performing architecture identified above (`rf_fit` by default, matching the NBA baseline's winner - swap this if the benchmark table above crowns a different model on real WNBA data).

Schedule caveat (also see issue #10's fix, referenced above under the round-robin note): the per-team matchup list generated below is a synthetic, randomly-drawn balanced schedule - each team gets a guaranteed floor of home meetings against every opponent plus a randomly assigned remainder - not the league's actual published fixture list for `SIMULATION_SEASON`. Real WNBA schedules are not balanced round-robins (they vary opponent counts, back-to-backs, and travel in ways this simulation does not model). Everything downstream that says "Projected Standings" is projecting win totals against this simulated

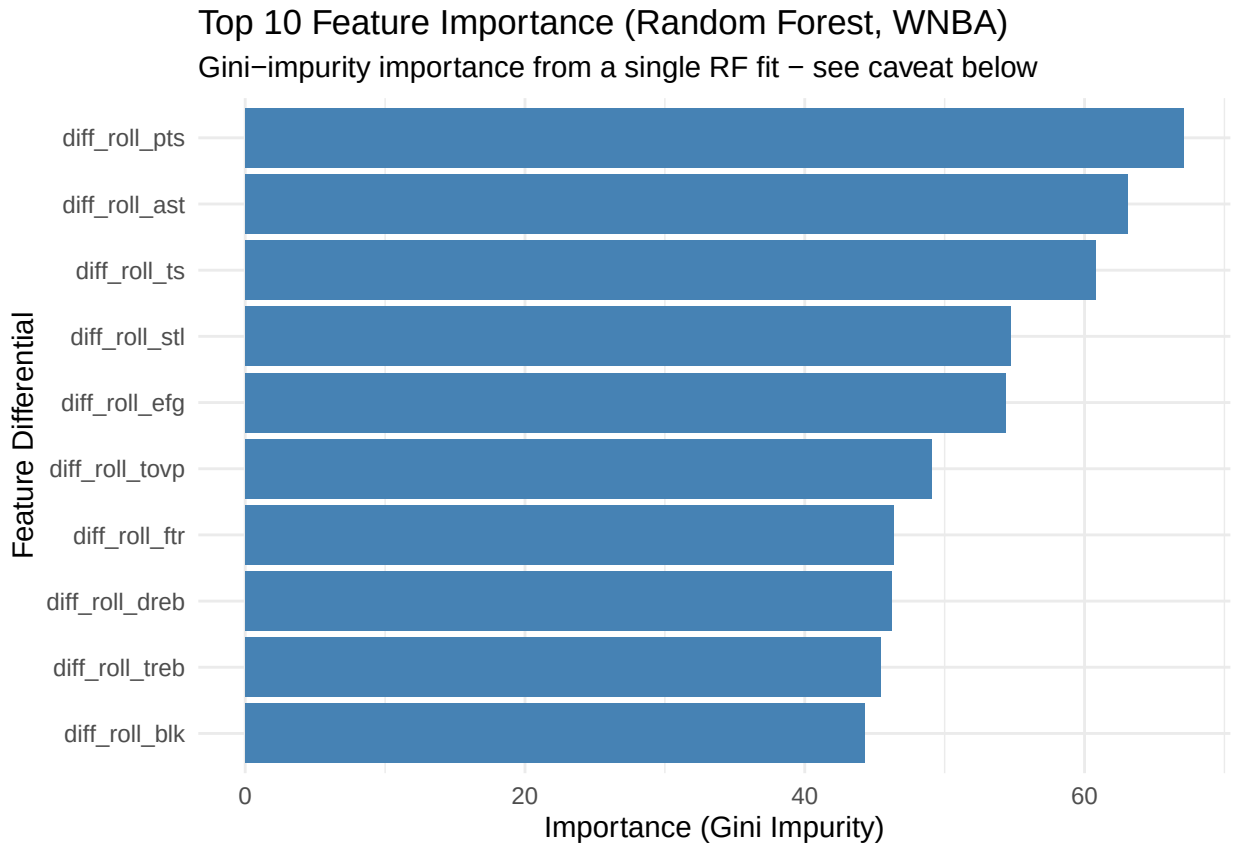


Figure 2: Top 10 features by Gini-impurity importance from the Random Forest model (rf_fit), ranked by mean decrease in node impurity.

schedule, not the real one, so calendar-specific effects (rest disparities tied to the actual slate, not just the `days_rest` placeholder set below) are not captured.

```
team_lookup <- box_scores |>
  select(team_id, team_name = team_short_display_name) |>
  distinct() |>
  mutate(team_id = as.integer(team_id))

current_season_df <- model_df |> filter(season == CURRENT_SEASON)

team_profiles_current <- bind_rows(
  current_season_df |>
    select(team_id = home_team, game_date, days_rest = home_days_rest, starts_with("home_roll_")) |>
    rename_with(~ sub("^home_roll_", "roll_", .), starts_with("home_roll_")),
  current_season_df |>
    select(team_id = away_team, game_date, days_rest = away_days_rest, starts_with("away_roll_")) |>
    rename_with(~ sub("^away_roll_", "roll_", .), starts_with("away_roll_"))
) |>
  group_by(team_id) |>
  slice_max(game_date, n = 1, with_ties = FALSE) |>
  ungroup()

teams <- unique(team_lookup$team_id)
set.seed(42)

n_teams <- length(teams)
half_season <- WNBA_SEASON_GAMES / 2  # e.g. 22 home / 22 away for a 44-game season

# Issue #27: replaced the base-R list-accumulator loop (sched_list <- list(); for (i in
# 1:n_teams) sched_list[[i]] <- ...; bind_rows(sched_list)) with purrr::map_dfr(), iterating
# 1:n_teams in the same order so the sample() draws below consume the RNG stream identically
# under set.seed(42). Per-team balanced-round-robin logic (issue #10's fix) is unchanged:
# every opponent gets `half_season %/% length(others)` guaranteed home meetings, and the
# `half_season %% length(others)` leftover meetings go to a randomly chosen subset of
# opponents each time this chunk runs.
sched_sim <- purrr::map_dfr(1:n_teams, function(i) {
  t_home <- teams[i]
  others <- setdiff(teams, t_home)
  base_reps <- half_season %/% length(others)
  remainder <- half_season %% length(others)
  extra_opps <- if (remainder > 0) sample(others, remainder) else integer(0)
  opps <- sample(c(rep(others, base_reps), extra_opps))
  tibble(
    home_team = t_home,
    away_team = opps,
    game_date = Sys.Date() + 1:half_season
  )
}) |>
  mutate(
    game_id = row_number(),
    season = SIMULATION_SEASON,
    home_win = FALSE,
    away_win = FALSE,
    home_days_rest = 2,
```

```
    away_days_rest = 2,
    diff_days_rest = 0
  )

profiles <- team_profiles_current |>
  select(team_id, starts_with("roll_"))

sched_sim_features <- sched_sim |>
  left_join(profiles, by = c("home_team" = "team_id")) |>
  rename_with(~ paste0("home_", .), starts_with("roll_")) |>
  left_join(profiles, by = c("away_team" = "team_id")) |>
  rename_with(~ paste0("away_", .), starts_with("roll_")) |>
  mutate(
    diff_roll_pts   = home_roll_pts - away_roll_pts,
    diff_roll_oreb  = home_roll_oreb - away_roll_oreb,
    diff_roll_dreb  = home_roll_dreb - away_roll_dreb,
    diff_roll_treb  = (home_roll_oreb + home_roll_dreb) - (away_roll_oreb + away_roll_dreb),
    diff_roll_ast   = home_roll_ast - away_roll_ast,
    diff_roll_stl   = home_roll_stl - away_roll_stl,
    diff_roll_blk   = home_roll_blk - away_roll_blk,
    diff_roll_tov   = home_roll_tov - away_roll_tov,
    diff_roll_efg   = home_roll_efg - away_roll_efg,
    diff_roll_tovp  = home_roll_tovp - away_roll_tovp,
    diff_roll_ftr   = home_roll_ftr - away_roll_ftr,
    diff_roll_ts    = home_roll_ts - away_roll_ts,
    diff_days_rest  = 0,
    target_home_win = factor("Win", levels = c("Win", "Loss"))
  )

predictions_sim <- sched_sim_features |>
  bind_cols(predict(rf_fit, sched_sim_features, type = "prob"))

home_var <- predictions_sim |>
  group_by(team_id = home_team) |>
  summarise(home_wins = sum(.pred_Win), home_var = sum(.pred_Win * (1 - .pred_Win)))

away_var <- predictions_sim |>
  group_by(team_id = away_team) |>
  summarise(away_wins = sum(.pred_Loss), away_var = sum(.pred_Loss * (1 - .pred_Loss)))

analytical_ci <- home_var |>
  left_join(away_var, by = "team_id") |>
  left_join(team_lookup, by = "team_id") |>
  mutate(
    Projected_Wins = round(home_wins + away_wins, 1),
    Projected_Losses = round(WNBA_SEASON_GAMES - Projected_Wins, 1),
    SE = round(sqrt(home_var + away_var), 2),
    CI_Lower_95 = round(Projected_Wins - 1.96 * SE, 1),
    CI_Upper_95 = round(Projected_Wins + 1.96 * SE, 1)
  ) |>
  arrange(desc(Projected_Wins)) |>
  mutate(Rank = row_number()) |>
  select(Rank, Team = team_name, Projected_Wins, Projected_Losses, SE, CI_Lower_95, CI_Upper_95)
```

```
knitr::kable(head(analytical_ci, 10), caption = sprintf("Top 10 Projected %d WNBA Standings with Analyti
```

Table 7: Top 10 Projected 2027 WNBA Standings with Analytical 95% CIs (against a synthetic simulated schedule, not the real published fixture list)

Rank	Team	Projected_Wins	Projected_Losses	SE	CI_Lower_95	CI_Upper_95
1	Lynx	31.2	12.8	3.17	25.0	37.4
2	Fever	27.6	16.4	2.81	22.1	33.1
3	Liberty	26.9	17.1	3.05	20.9	32.9
4	Dream	26.7	17.3	3.07	20.7	32.7
5	Valkyries	24.9	19.1	3.24	18.5	31.3
6	Mercury	23.4	20.6	3.13	17.3	29.5
7	Mystics	22.9	21.1	3.16	16.7	29.1
8	Wings	22.8	21.2	3.08	16.8	28.8
9	Fire	21.0	23.0	3.06	15.0	27.0
10	Aces	20.4	23.6	3.08	14.4	26.4

Monte Carlo Simulation (`N_SIMULATIONS` Seasons)

```
set.seed(42)
n_sims <- N_SIMULATIONS
n_games <- nrow(predictions_sim)

sim_matrix <- matrix(rbinom(n_games * n_sims, 1, predictions_sim$.pred_Win), nrow = n_games, ncol = n_si

home_teams_vec <- predictions_sim$home_team
away_teams_vec <- predictions_sim$away_team

teams_vec <- unique(team_lookup$team_id)

# Issue #26: sapply()'s return type/shape depends on runtime simplification rules; vapply()
# with an explicit FUN.VALUE is type-stable and guaranteed to produce the same
# n_sims x length(teams_vec) matrix shape the as.vector() call below expects.
team_wins_matrix <- vapply(teams_vec, function(t_id) {
  colSums(sim_matrix[home_teams_vec == t_id, , drop = FALSE]) +
  colSums((1 - sim_matrix)[away_teams_vec == t_id, , drop = FALSE])
}, FUN.VALUE = numeric(n_sims))

sim_results <- tibble(
  sim = rep(1:n_sims, times = length(teams_vec)),
  team_id = rep(teams_vec, each = n_sims),
  total_wins = as.vector(team_wins_matrix)
)

standings_sim_summary <- sim_results |>
  left_join(team_lookup, by = "team_id") |>
  group_by(Team = team_name) |>
  summarise(
    Mean_Wins = round(mean(total_wins), 1),
    Median_Wins = median(total_wins),
```

```
    `5th_Pct` = quantile(total_wins, 0.05),
    `95th_Pct` = quantile(total_wins, 0.95),
    `2.5th_Pct` = quantile(total_wins, 0.025),
    `97.5th_Pct` = quantile(total_wins, 0.975)
  ) |>
  arrange(desc(Mean_Wins)) |>
  mutate(Rank = row_number()) |>
  select(Rank, Team, Mean_Wins, Median_Wins, `5th_Pct`, `95th_Pct`, `2.5th_Pct`, `97.5th_Pct`)

knitr::kable(head(standings_sim_summary, 10), caption = sprintf("Top 10 Projected %d WNBA Standings fro
```

Table 8: Top 10 Projected 2027 WNBA Standings from 10000 Monte Carlo Simulations (against a synthetic simulated schedule, not the real published fixture list)

Rank	Team	Mean_Wins	Median_Wins	5th_Pct	95th_Pct	2.5th_Pct	97.5th_Pct
1	Lynx	31.2	31	26	36	25	37
2	Fever	27.6	28	23	32	22	33
3	Liberty	26.9	27	22	32	21	33
4	Dream	26.7	27	21	32	20	32
5	Valkyries	24.9	25	20	30	19	31
6	Mercury	23.4	23	18	28	17	29
7	Mystics	22.9	23	18	28	17	29
8	Wings	22.8	23	18	28	17	29
9	Fire	21.0	21	16	26	15	27
10	Aces	20.4	20	15	25	15	27

Win Total Density Ridge Plot

```
top10_teams <- head(standings_sim_summary$Team, 10)

sim_results |>
  left_join(team_lookup, by = "team_id") |>
  filter(team_name %in% top10_teams) |>
  ggplot(aes(x = total_wins, y = reorder(team_name, total_wins, median), fill = team_name)) +
  geom_density_ridges(alpha = 0.7, show.legend = FALSE) +
  # Issue #39: default ggplot2 categorical hue palette is not colorblind-safe; viridis's
  # discrete scale is perceptually uniform and distinguishable under the common forms of color
  # vision deficiency (viridisLite is already a transitive dependency of ggplot2/scales per
  # renv.lock, so no new pinned package is required).
  scale_fill_viridis_d() +
  geom_vline(xintercept = WNBA_SEASON_GAMES / 2, linetype = "dashed", color = "darkred") +
  annotate(
    geom = "label",
    x = WNBA_SEASON_GAMES / 2,
    y = Inf,
    label = ".500 win pct.",
    color = "darkred",
    fill = "white",
    vjust = 1.2,
    hjust = -0.05,
    size = 3,
```

```
    label.size = 0
  ) +
  theme_minimal() +
  labs(
    title = "Simulated Win Total Distributions (WNBA Monte Carlo)",
    subtitle = sprintf("Top 10 Projected Teams for %d Season", SIMULATION_SEASON),
    x = sprintf("Total Wins (%d-Game Season)", WNBA_SEASON_GAMES),
    y = "Team"
  )
```

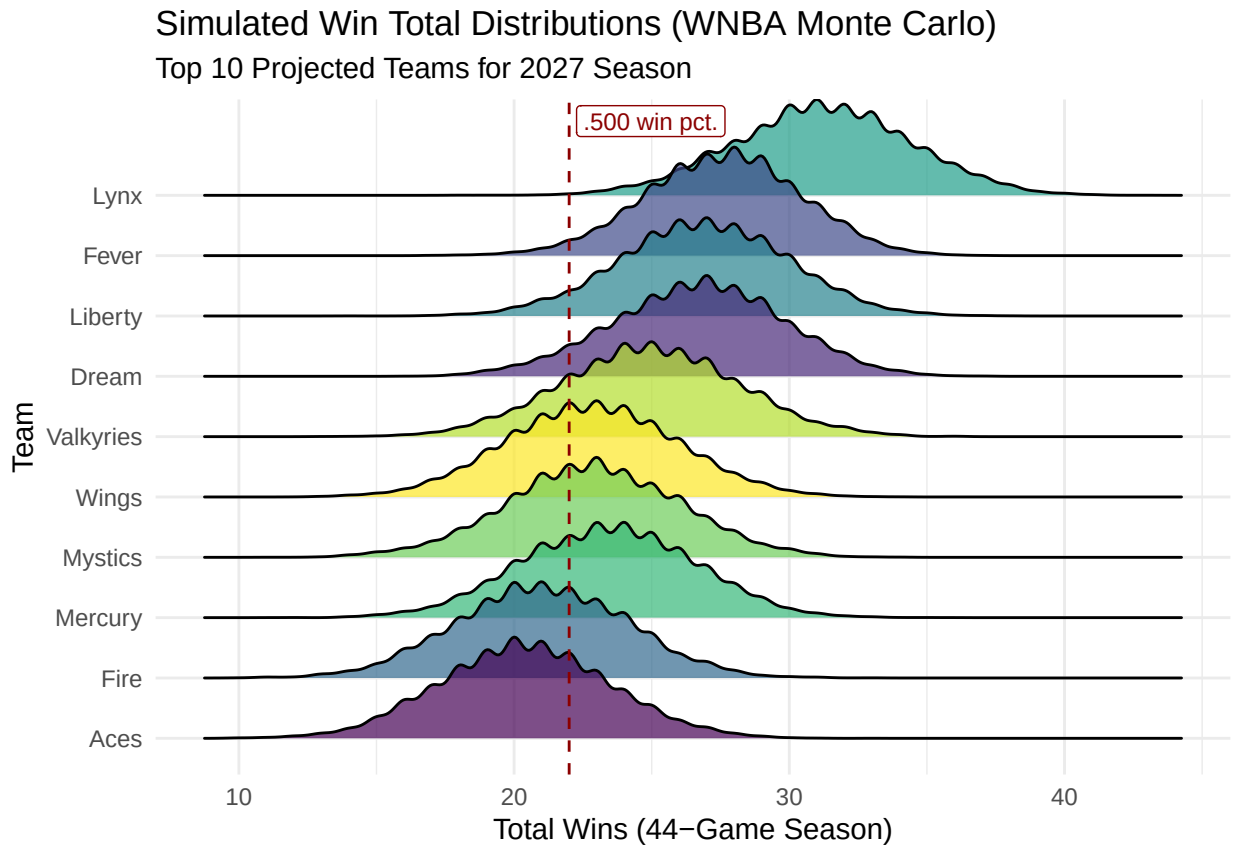


Figure 3: Simulated win-total distributions for the top 10 projected WNBA teams across all Monte Carlo season simulations, with the .500 win-percentage threshold marked.

Conclusion & Next Steps

Returning to the question posed in the Introduction - does the WNBA's shorter season change the NBA baseline's modeling choices, and what does that mean for projected standings - the short answer is "partly." On rolling-window size, the WNBA does need a different value than a naive proportional scaling of the NBA's 18-game window suggested: a full 3-25 sweep on real WNBA data found the empirical peak at window size 12, not the placeholder of 9 originally carried over, though the sweep is noisy enough (windows 12/13/15 within 0.003-0.015 ROC-AUC of each other) that this should be read as the best point in a single-split sweep rather than a sharply confirmed optimum. On the offseason-transition method, the NBA's answer *did* transfer: "reset" beat both `mean_regress` and `decay` on ROC-AUC and Brier score when re-tested on WNBA data, so `config.R`'s `OFFSEASON_METHOD` default stays `"reset"`. On model architecture, the result is mixed rather than a clean transfer: Random Forest, the NBA baseline's winner, still wins on Brier score,

Log-Loss, and Accuracy against WNBA data, but XGBoost narrowly edges it on ROC-AUC - a gap small enough to be plausibly noise (see the Model Benchmarking section above) rather than a reversal of which architecture is "really" better. Random Forest (`rf_fit`) is retained as the production model on the strength of its calibration-metric wins.

Feeding that retained model into a `WNBA_SEASON_GAMES`-game Monte Carlo season simulation produces the win-total and standings projections in the Simulation-Season and Monte Carlo sections above: at the time of this render, the Minnesota Lynx project as the top team, with a mean simulated win total around 31 of 44 games (95% interval roughly 25-37), followed closely by the Indiana Fever and New York Liberty in the upper-20s. (This ordering is a live simulation output, not a fixed number - it can shift slightly between renders as the underlying data and random draws change; see Tables 4 and 5 for the full, currently-rendered top-10 projected standings with both analytical and simulation-based confidence intervals.)

In short: a WNBA-specific rolling-window size and a re-validated (if narrowly mixed) model choice both mattered enough to re-derive rather than assume, while the offseason-transition method turned out to carry over from the NBA unchanged. All three choices are now backed by real re-benchmarks on WNBA data rather than proportional-scaling guesses, and `config.R`'s live constants (`ROLLING_WINDOW_SIZE`, `OFFSEASON_METHOD`) reflect the winners above.

Process notes, for anyone maintaining this analysis going forward: this notebook has been knit end-to-end (see the provenance note at the top of this document), so every table above is real render output, not hand-pasted. If the underlying data or either constant's empirical winner changes again, the relevant experiment chunk(s) above should be re-run, `config.R` updated accordingly, and that change logged in `task_assignments.md`; this file should keep the same "pristine baseline, do not hand-edit" discipline as `NBA_Analysis_Project.Rmd` (`ADR.md` Principle I) - re-run the whole methodology fresh via `rmarkdown::render()` rather than patching numbers in by hand. For downstream player-level rating work (bottom-up team ratings, lineup synergy, roster continuity), see `roster_bottom_up_engine.R`, `lineup_synergy.R`, and `roster_dynamics.R` in this same folder - those were built independently of this notebook's team-level rolling-feature model and read `WNBA_MARGIN_SIGMA` (`config.R`) rather than this notebook's classifier probabilities for their own margin-to-win-probability conversion.